AI ASSISTED CODING

ASSIGNMENT-22

T.HimaBindhu

2303A52050

Batch:38

Lab 22 – Advanced Ethical Considerations: Addressing Complex Scenarios and Limitations

Task 1 – Bias Detection in AI Suggestions (Alternative Example)

Example Scenario:

Ask AI to generate a resume shortlisting algorithm multiple times.

• Change prompt wording (e.g., “select best candidate” vs “ensure diversity”).

• Compare outputs and logic.

Discussion Points:

• Does the AI favor certain qualifications, backgrounds, or experiences?

• How could biased shortlisting algorithms affect equal employment opportunities?

Prompt:

Generate a resume shortlisting algorithm in Python to select the best candidate based on qualifications and experience.

Code & Output:

```python
# Bias Detection in AI Suggestions (Alternative Example)
def detect_bias(suggestions, sensitive_attributes):
    bias_report = {}
    for attribute in sensitive_attributes:
        bias_report[attribute] = {}
        for suggestion in suggestions:
            if attribute in suggestion:
                bias_report[attribute][suggestion] = bias_report[attribute].get(suggestion, 0) + 1
    return bias_report
if __name__ == "__main__":
    suggestions = [
        "The best candidate for the job is a young man",
        "The best candidate for the job is a young woman",
        "The best candidate for the job is an experienced professional"
    ]
    sensitive_attributes = ["young", "man", "woman", "experienced"]
    bias_report = detect_bias(suggestions, sensitive_attributes)
    print(bias_report)
```

```
{'young': {'The best candidate for the job is a young man': 1, 'The best candidate for the job is a young woman': 1}, 'man': {'The best candidate for the job is
a young man': 1, 'The best candidate for the job is a young woman': 1}, 'woman': {'The best candidate for the job is a young woman': 1}, 'experienced': {'The bes
t candidate for the job is an experienced professional': 1}}
```

Explanation:

In this task, AI is used to generate resume screening algorithms under different prompt conditions to observe potential bias. When asked to select the “best candidate,” the AI may prioritize factors like higher education, specific institutions, or years of experience, which can unintentionally favor certain groups. However, when the prompt emphasizes diversity and fairness, the algorithm may include balanced criteria and avoid over-prioritizing narrow qualifications. Comparing these outputs helps identify how prompt wording influences AI decisions and highlights the risk of bias in automated hiring systems.

Task 2 – Privacy and Sensitive Data Handling (Alternative Example)

Example Scenario:

Ask AI to write code for processing student academic records (roll number, marks, email).

Discussion Points:

• How should student identity data be masked or encrypted?

• What privacy laws (e.g., FERPA, GDPR-like principles) are relevant?

• What ethical responsibility does the programmer have when using AI-generated code?

Prompt:

Generate a Python program to process student academic records including roll number, marks, and email, and then enhance it by applying data masking, encryption techniques, and secure handling practices to protect sensitive information.

Code & Output:

```python
# Privacy and Sensitive Data Handling (Alternative Example)
def handle_sensitive_data(data, sensitive_fields):
    for field in sensitive_fields:
        if field in data:
            data[field] = "REDACTED"
    return data
if __name__ == "__main__":
    user_data = {
        "name": "John Doe",
        "email": "john.doe@example.com",
        "ssn": "123-45-6789"
    }
    sensitive_fields = ["email", "ssn"]
    processed_data = handle_sensitive_data(user_data, sensitive_fields)
    print(processed_data)
```

```
{'name': 'John Doe', 'email': 'REDACTED', 'ssn': 'REDACTED'}
```

Explanation:

In this task, AI is used to create a program that handles sensitive student data, which raises important privacy concerns. Personal information such as roll numbers and emails should be masked (e.g., partially hidden) or encrypted to prevent unauthorized access. Privacy regulations like FERPA and General Data Protection Regulation emphasize protecting personal data and ensuring it is used responsibly. Programmers must ensure that AI-generated code follows secure coding practices, avoids exposing sensitive data, and complies with legal and ethical standards.

Task 3 – AI-Generated Security Risks (Alternative Example)

Example Scenario:

Use AI to generate a payment processing script for an online shopping system.

Student Tasks:

• Identify risks such as:

o Plain-text card data storage

o Missing input validation

• Propose ethical coding standards to prevent financial fraud and misuse.

Prompt:

Generate a Python-based payment processing script for an online shopping system, then analyze it to identify security risks such as plain-text card data storage and lack of input validation, and improve it using secure coding practices.

Code & Output:

```python
# AI-Generated Security Risks (Alternative Example)
def identify_security_risks(ai_output):
    risks = []
    if "password" in ai_output.lower():
        risks.append("Exposure of sensitive information")
    if "sql injection" in ai_output.lower():
        risks.append("Vulnerability to SQL injection attacks")
    if "cross-site scripting" in ai_output.lower():
        risks.append("Vulnerability to cross-site scripting (XSS) attacks")
    return risks
if __name__ == "__main__":
    ai_output = "The user entered their password and is vulnerable to SQL injection."
    risks = identify_security_risks(ai_output)
    print("Identified Security Risks:", risks)
```

```
Identified Security Risks: ['Exposure of sensitive information', 'Vulnerability to SQL injection attacks']
```

Explanation:

In this task, AI-generated code is evaluated for potential security vulnerabilities in handling financial data. Storing card details in plain text or failing to validate user inputs can lead to serious risks like data breaches and fraud. Ethical coding standards require developers to use encryption, secure APIs, and proper validation techniques to protect sensitive information. Additionally, compliance with standards like PCI DSS ensures safe handling of payment data.

Task 4 – Accountability in Human–AI Collaboration (Alternative Example)

Scenario:

AI generates code for an autonomous traffic signal control system.

Discussion Questions:

• If a software error causes accidents, who is accountable?

• Should responsibility lie with:

o AI tool developers

o Software engineers

o Government authorities deploying the system?

Students submit a short reflection on accountability.

Prompt:

Generate code for an autonomous traffic signal control system using AI, then analyze accountability if a software failure leads to accidents, considering the roles of developers, engineers, and authorities.

Code & Output:

```
# Accountability in Human-AI Collaboration (Alternative Example)
def log_ai_decision(user_input, ai_output):
    log_entry = f"User Input: {user_input}\nAI Output: {ai_output}\n"
    with open("ai_decision_log.txt", "a") as log_file:
        log_file.write(log_entry)
if __name__ == "__main__":
    user_input = "What is the capital of France?"
    ai_output = "The capital of France is Paris."
    log_ai_decision(user_input, ai_output)
    print("AI decision logged successfully.")
```

```
AI decision logged successfully.
```

Explanation:

This task explores responsibility in human–AI collaboration, especially in safety-critical systems. If an error in an AI-generated traffic system causes accidents, accountability cannot rest on a single entity alone. AI tool developers, software engineers, and government authorities all share responsibility at different levels, from design to deployment. Engineers must validate and test the system thoroughly before use, while authorities must ensure proper regulations and approvals.

Task 5 – Limitations of AI in Critical Systems (Alternative Example)

Example Scenario:

Ask AI to generate code for an aircraft maintenance fault detection system.

Discussion Points:

• Can AI-generated code be trusted without expert validation?

• What human oversight is essential in aviation software?

• How should safety-critical code be reviewed and certified?

Prompt:

Ask AI to generate code for an aircraft maintenance fault detection system, then evaluate its reliability, required human oversight, and certification standards for safety-critical software.

Code & Output:

```
# Limitations of AI in Critical Systems (Alternative Example)
def evaluate_ai_decision(ai_output):
    if "critical" in ai_output.lower():
        return "AI decision may not be reliable for critical systems."
    else:
        return "AI decision is acceptable for non-critical systems."
if __name__ == "__main__":
    ai_output = "The AI has made a critical decision regarding the system's operation."
    evaluation = evaluate_ai_decision(ai_output)
    print(evaluation)
```

```
AI decision may not be reliable for critical systems.
```

Explanation:

AI-generated code in aviation systems cannot be blindly trusted due to the high risk involved. Systems like aircraft fault detection require rigorous validation, expert review, and compliance with strict aviation safety standards. Human oversight from domain experts, engineers, and regulatory bodies is essential to verify correctness and reliability. The software must undergo extensive testing, simulation, and certification processes before deployment. This task demonstrates the limitations of AI and the necessity of human expertise in critical systems.

Task 6 – Ethical Use of AI-Generated Code (Alternative Example)

Example Scenario:

Students analyze cases where AI-generated code is used in academic assignments or startups.

Discussion Points:

• Risks of submitting AI code without understanding it

• Copyright and license violations

Best practices:

o Proper citation

o Manual verification

o Ethical reuse of AI outputs

Prompt:

Analyze the use of AI-generated code in academic projects or startups, focusing on ethical concerns such as understanding, citation, and responsible reuse of generated outputs.

Code & Output:

```
# Ethical Use of AI-Generated Code (Alternative Example)
def evaluate_ai_code(ai_code):
    if "unethical" in ai_code.lower():
        return "The AI-generated code may contain unethical content."
    else:
        return "The AI-generated code appears to be ethical."
if __name__ == "__main__":
    ai_code = "This AI-generated code contains unethical practices."
    evaluation = evaluate_ai_code(ai_code)
    print(evaluation)
```

```
The AI-generated code may contain unethical content.
```

Explanation:

This task emphasizes ethical considerations when using AI-generated code. Submitting code without understanding it can lead to learning gaps and potential errors in real-world applications. There are also risks of copyright or license violations if AI outputs are reused without proper attribution. Best practices include verifying the code manually, citing AI assistance where required, and ensuring ethical reuse.